

Python Object-Oriented Programming (OOP) Deep Dive

"The four pillars of OOP are Encapsulation, Abstraction, Inheritance, and Polymorphism."

CREATED FOR CAREER

By Amar Panchal, Technical Trainer and Co-founder of Career Credentials.
17 years of expertise in coding and placement training for freshers.

Object-Oriented Programming (OOP) is a paradigm based on the concept of "objects," which can contain data and code (attributes and methods). It aims to implement real-world entities like inheritance, hiding, and polymorphism in programming.

1. Classes and Objects

Theory and Syntax

Class (Blueprint): A class is a blueprint or a template for creating objects. It defines a set of attributes (data/variables) and methods (functions) that the objects will possess.

Object (Instance): An object is a specific instance of a class. When a class is defined, no memory is allocated until an object is created.

- `class` `ClassName::` Keyword used to define a class.
- `__init__(self, ...)`: The special "constructor" method that is automatically called when a new object is created. It's used to initialize the object's attributes.
- `self`: Represents the instance of the class itself. It must be the first parameter of any instance method, and it is used to access the instance's attributes and methods.

Concept	Syntax
Define Class	<code>class ClassName:</code>
Constructor	<code>def __init__(self, arg1, arg2):</code>
Create Object	<code>obj_name = ClassName(value1, value2)</code>

Simple Example

```
class Student:
    """A blueprint for creating student objects."""

    # 1. Constructor method to initialize attributes
    def __init__(self, name, roll_number):
        self.name = name # Attribute 1
        self.roll_number = roll_number # Attribute 2

    # 2. Instance method (behavior)
    def display_info(self):
        print(f"Student Name: {self.name}, Roll Number: {self.roll_number}")

# Creating objects (instances) of the Student class
student1 = Student("Priya Sharma", 101)
student2 = Student("Karan Singh", 102)

# Accessing methods
print("--- Classes and Objects Example ---")
student1.display_info()
student2.display_info()
# Output:
# Student Name: Priya Sharma, Roll Number: 101
# Student Name: Karan Singh, Roll Number: 102
```

2. Encapsulation

Theory and Syntax

Encapsulation is the process of wrapping data (attributes) and the methods that operate on that data into a single unit (the class). It also involves restricting direct

access to some of the object's components, preventing external, unmanaged access to sensitive data.

In Python, we use naming conventions to indicate intent: Public, Protected (`_attribute`), and Private (`__attribute` via name mangling).

Access Level	Naming Convention	External Access
Public	<code>attribute</code>	Direct access allowed.
Protected	<code>_attribute</code>	Access is discouraged, but possible.
Private	<code>__attribute</code>	Access is highly restricted via Name Mangling.

Simple Example

```
class BankAccount:
    """Demonstrates encapsulation of the account balance."""

    def __init__(self, initial_balance):
        # Private attribute: should only be modified via controlled
        # methods
        self.__balance = initial_balance

        # Method to allow controlled modification of the private
        # attribute (Setter)
        def deposit(self, amount):
            if amount > 0:
                self.__balance += amount
                print(f"Deposited ${amount}. New Balance:
                ${self.__balance}")
            else:
                print("Deposit amount must be positive.")

        # Method to allow controlled reading of the private attribute
        # (Getter)
        def get_balance(self):
            # Data is only revealed through a safe, public method
            return self.__balance

# Create an account
```

```
my_account = BankAccount(1000)

print("\n--- Encapsulation Example ---")
# Accessing via public method (GOOD PRACTICE)
my_account.deposit(200)
print(f"Current Balance (via method): ${my_account.get_balance()}")

# Attempting to access the private attribute directly (Will likely fail or be non-obvious)
# This demonstrates that direct access is prevented/discouraged.
try:
    print(f"Direct Access Attempt: {my_account.__balance}")
except AttributeError as e:
    print(f"Direct access failed: {e}")
```

3. Inheritance

Theory and Syntax

Inheritance is a mechanism where a new class (the **subclass** or **child class**) is derived from an existing class (the **superclass** or **parent class**). The child class inherits all the attributes and methods of the parent class, allowing for code reuse and the creation of a hierarchy.

- `class ChildClass(ParentClass)::` Syntax for defining a class that inherits from another class.
- `super().__init__(...):` Used within the child's constructor to call the parent class's constructor.

Concept	Syntax
Subclass Definition	<code>class Subclass(Superclass):</code>
Call Parent's Constructor	<code>super().__init__(args)</code>

Simple Example

```
class Vehicle: # Parent Class
    """A generic vehicle with a maximum speed."""
    def __init__(self, max_speed):
        self.max_speed = max_speed

    def start_engine(self):
```

```

        return "Engine started."

class Car(Vehicle): # Child Class, inherits from Vehicle
    """A specific type of vehicle with a model attribute."""
    def __init__(self, max_speed, model):
        # Call the parent's constructor to initialize max_speed
        super().__init__(max_speed)
        self.model = model

    def drive(self):
        return f"The {self.model} is driving at max speed {self.max_speed} km/h."

# Create a Car object
my_car = Car(220, "Sedan X")

print("\n--- Inheritance Example ---")
# Car object can access the start_engine method from the Vehicle class
print(my_car.start_engine())
print(my_car.drive())
# Output:
# Engine started.
# The Sedan X is driving at max speed 220 km/h.

```

3.1. Types of Inheritance in Python

Python supports four main types of inheritance, allowing developers great flexibility in structuring their code.

1. Single Inheritance

Theory: A child class inherits from only one parent class. This is the simplest and most common form of inheritance.

Process: ChildClass inherits features from ParentClass.

```

class Animal: # Parent
    def eat(self):
        print("Animal is eating.")

class Dog(Animal): # Child (inherits from Animal)
    def bark(self):
        print("Dog is barking.")

print("\n--- Single Inheritance Example ---")
dog = Dog()

```

```
dog.eat() # Inherited method
dog.bark() # Own method
# Output:
# Animal is eating.
# Dog is barking.
```

2. Multiple Inheritance

Theory: A child class inherits from two or more parent classes. Python allows this, but developers must be aware of the Method Resolution Order (MRO).

Process: ChildClass inherits features from both ParentClass1 and ParentClass2.

```
class Swimmer: # Parent 1
    def swim(self):
        print("Can swim.")

class Walker: # Parent 2
    def walk(self):
        print("Can walk.")

class Human(Swimmer, Walker): # Child (inherits from two parents)
    def think(self):
        print("Can think.")

print("\n--- Multiple Inheritance Example ---")
person = Human()
person.swim() # Inherited from Swimmer
person.walk() # Inherited from Walker
person.think() # Own method
# Output:
# Can swim.
# Can walk.
# Can think.
```

3. Multilevel Inheritance

Theory: Features are inherited from one parent class, which itself is inherited from another parent class, forming a chain.

Process: GrandChildClass inherits from ChildClass, which in turn inherits from ParentClass.

```
class Grandparent: # Grandparent
    def property(self):
        print("Has some inherited property.")
```

```

class Parent(Grandparent): # Parent (inherits from Grandparent)
    def profession(self):
        print("Has a specific profession.")

class Child(Parent): # Child (inherits from Parent)
    def study(self):
        print("Is currently studying.")

print("\n--- Multilevel Inheritance Example ---")
kid = Child()
kid.property() # Inherited from Grandparent
kid.profession() # Inherited from Parent
kid.study() # Own method
# Output:
# Has some inherited property.
# Has a specific profession.
# Is currently studying.

```

4. Hierarchical Inheritance

Theory: Multiple child classes inherit from a single parent class, creating specialized versions.

Process: ChildClass1 and ChildClass2 both inherit features from a single ParentClass.

```

class Shape: # Parent
    def draw(self):
        print("Drawing a geometric shape.")

class Circle(Shape): # Child 1 (inherits from Shape)
    def area(self):
        print("Calculating circle area.")

class Square(Shape): # Child 2 (inherits from Shape)
    def perimeter(self):
        print("Calculating square perimeter.")

print("\n--- Hierarchical Inheritance Example ---")
c = Circle()
s = Square()

c.draw() # Inherited from Shape
c.area() # Own method

s.draw() # Inherited from Shape
s.perimeter() # Own method

```

```
# Output:
# Drawing a geometric shape.
# Calculating circle area.
# Drawing a geometric shape.
# Calculating square perimeter.
```

4. Polymorphism

Theory and Syntax

Polymorphism (Greek for "many forms") allows different objects to respond to the same method call in their own way. In Python, this is primarily achieved through **Method Overriding** and **Duck Typing**.

Simple Example (Duck Typing)

```
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

class Duck:
    def speak(self):
        return "Quack!"

# A function that expects any object that has a 'speak' method
def animal_action(animal):
    """The function doesn't care about the object's type,
    only that it has a 'speak' method."""
    print(f"The animal says: {animal.speak()}")

# Create objects
print("\n--- Polymorphism (Duck Typing) Example ---")
dog_obj = Dog()
cat_obj = Cat()
duck_obj = Duck()

# The same function call (animal_action) produces different results
animal_action(dog_obj)
animal_action(cat_obj)
animal_action(duck_obj)

# Output:
```

```
# The animal says: Woof!
# The animal says: Meow!
# The animal says: Quack!
```

5. Abstraction

Theory and Syntax

Abstraction involves simplifying complexity by hiding the implementation details and showing only the essential information to the user. This is achieved by defining an **Abstract Base Class (ABC)** which cannot be instantiated and must have at least one **Abstract Method**.

- `from abc import ABC, abstractmethod`: Imports necessary tools.
- `class MyAbstractClass(ABC):`: The abstract class must inherit from ABC.
- `@abstractmethod`: Decorator placed above a method to declare it as abstract.

Concept	Syntax
Import	<code>from abc import ABC, abstractmethod</code>
Define Abstract Class	<code>class MyClass(ABC):</code>
Define Abstract Method	<code>@abstractmethod</code> <code>def my_method(self):</code>

Simple Example

```
from abc import ABC, abstractmethod

class NotificationService(ABC): # Inherits from ABC
    """Abstract Base Class for any notification service."""

    @abstractmethod # This method MUST be implemented by all
    subclasses
    def send_message(self, recipient, message):
        pass # No implementation detail here

    def log_status(self):
        # A concrete method that all subclasses can use
        print("Logging notification status...")
```

```

class EmailNotifier(NotificationService): # Concrete subclass
    """A concrete implementation for sending emails."""

    # REQUIRED: Implementation of the abstract method
    def send_message(self, recipient, message):
        print(f"Sending EMAIL to {recipient}: '{{message}}'")

class SMSNotifier(NotificationService): # Another concrete subclass
    """A concrete implementation for sending SMS."""

    # REQUIRED: Implementation of the abstract method
    def send_message(self, recipient, message):
        print(f"Sending SMS to {recipient}: '{{message}}'")

# Create concrete objects
email_service = EmailNotifier()
sms_service = SMSNotifier()

print("\n--- Abstraction Example ---")
email_service.send_message("user@example.com", "Your order has shipped.")
sms_service.send_message("9876543210", "New promotion alert!")
email_service.log_status()

# Attempting to create an object of the abstract class will raise an error
try:
    abstract_service = NotificationService()
except TypeError as e:
    print(f"\nFailed to instantiate NotificationService (Abstract Class): {{e}}")

```

Connect with Amar Panchal

 Website
  LinkedIn
  YouTube
  Instagram
  Facebook

 9821601163

